

Pure Mathematical Decision Model

Chicago Bare-Metal CME Futures Microstructure System

Developer handoff specification for a model-agnostic, queue-aware, latency-aware decision engine using CME MBO/order-flow state, Rithmic live execution, Databento historical MBO event windows, and HftBacktest replay.

Executive Summary

The mathematically clean model is a regime-conditioned marked point process embedded inside a queue-aware stochastic-control decision engine. It is not a chart model and not a single hard-coded strategy. It values each possible action by expected net PnL after latency, queue position, fill probability, fees, slippage, adverse selection, inventory risk, and tail loss.

Core object:

- Regime-conditioned MBO marked point process
- + queue/fill probability model
- + adverse-selection model
- + constrained stochastic-control optimizer

Production mapping:

```
pi_theta : X_t -> {EV(a), P_fill(a), AS(a), ES(a), Q(a), size(a)} for all actions a
a*_t      = argmax_a Q(X_t, a)
n*_t      = approved_position_size(a*_t)
```

1. Probability Space and Information Set

Define a filtered probability space:

$(\Omega, \mathcal{F}, \{\mathcal{F}_t\}_{t \geq 0}, P)$

$\mathcal{F}_t$ = all information available up to decision time t only.

No model feature, target, label, or execution simulation may use information after t .

This prevents lookahead leakage and makes the decision problem a valid adapted process.

2. Full State Vector

At any time t , define the observable and latent state:

$X_t = (B_t, A_t, Q_t, I_t, Z_t, L_t, E_t, V_t)$

B_t = bid-side book state

A_t = ask-side book state

Q_t = queue-position / fill state

I_t = current inventory / position

Z_t = latent regime state

L_t = latency state

E_t = event context: CPI, NFP, FOMC, prop-flat, cash open, normal

V_t = volatility / liquidity state

Component	Mathematical role	Practical meaning
B_t, A_t	Discrete order-book queues	Depth at bid/ask levels, adds, cancels, trades, modifies
Q_t	Queue state	Queue-ahead volume and fill probability for candidate orders
I_t	Inventory state	Position, exposure, current risk budget
Z_t	Latent state	Normal, event shock, vacuum, stop cascade, prop flatten, chop, trend
L_t	Latency state	Observed and modeled delay distribution
E_t	Event conditioning	CPI/NFP/FOMC/open/forced-flat/Friday late session

Component	Mathematical role	Practical meaning
V_t	Risk/liquidity state	Realized vol, spread stress, depth stability

3. Limit Order Book Representation

$B_t = \{(p^b_i, q^b_i)\}$ for $i = 1, \dots, K$
 $A_t = \{(p^a_i, q^a_i)\}$ for $i = 1, \dots, K$

p^b_i = bid price at level i
 q^b_i = bid quantity at level i
 p^a_i = ask price at level i
 q^a_i = ask quantity at level i
 K = depth levels modeled

The model consumes the order book as a vector of queues and event transitions, not as candles.

4. MBO Marked Point Process

Each Level 3/MBO event is a marked event. Event type k may represent add, cancel, modify, trade, side, level, size bucket, or event context.

N^k_t = counting process for MBO event type k

$\lambda_k(t) = \exp(\beta_k' f(X_t) + \sum_j \int_0^t \phi_{kj}(t-s) dN^j_s)$
)

$\beta_k' f(X_t)$ = current-state effect
 $\sum_j \int_0^t \phi_{kj}(t-s) dN^j_s$ = self/cross excitation

This structure is appropriate because CME MBO updates are event-driven and irregularly spaced. A marked point process can represent cancel storms, trade bursts, queue depletion, and post-event clustering.

5. Latent Regime Filter

Z_t in {
 normal,
 event_shock,
 liquidity_vacuum,
 stop_cascade,
 prop_flatten,
 book_rebuild,
 chop,
 trend_continuation,
 spread_stress
 }

Required output:
 $P(Z_t = z | F_t)$ for every regime z

Implementation may use HMMs, state-space filters, Bayesian filters, Kalman-style filters where applicable, boosted models, neural filters, or ensembles. The required mathematical output is a regime posterior, not a single hard label.

6. Queue and Fill Probability

$H_t(a)$ = queue-ahead volume for candidate action a

$\tau_{\text{fill}}(a) = \inf\{u \geq t : H_u(a) \leq 0\}$

$P_{\text{fill}}(a, h) = P(\tau_{\text{fill}}(a) \leq t+h | X_t, a)$

A directional signal has no production value unless the system estimates whether the order can actually fill under queue position, latency, and order type.

7. Adverse Selection

```
For long fill:
    AS_h = (m_{t+h} - p_exec) / tick_size

For short fill:
    AS_h = (p_exec - m_{t+h}) / tick_size

m_t      = midprice at time t
p_exec   = execution price
h        = evaluation horizon: 100ms, 500ms, 1s, 5s
```

If expected adverse selection is negative after a passive fill, the fill is toxic: the order is likely being filled just before price moves against it.

8. Action Space

```
A_t = {
    no_trade,
    enter_long,
    enter_short,
    add,
    reduce,
    flatten,
    join_bid,
    join_ask,
    improve_bid,
    improve_ask,
    marketable_buy,
    marketable_sell,
    cancel,
    replace
}
```

The production model is an action-value model. It should compare the expected value of all valid actions, including no-trade.

9. Fill-Adjusted Expected Value

$$EV_t(a) = E[Pi_{t,h}(a) \mid F_t]$$

```
Pi_{t,h}(a) =
    PnL_gross(a)
    - fees(a)
    - slippage(a)
    - adverse_selection(a)
    - missed_fill_cost(a)
    - latency_cost(a)
```

Expanded:

```
EV_t(a) =
    P_fill(a) * E[PnL | fill, a]
    - (1 - P_fill(a)) * C_miss(a)
    - C_fees(a)
    - C_latency(a)
    - C_slippage(a)
```

A trade is not valid because direction probability is high. A trade is valid only if fill-adjusted expected value is positive after costs and risk penalties.

10. Risk-Adjusted Optimization

```
a*_t = argmax_{a in A_t} [
    EV_t(a)
    - lambda_AS * AS_risk_t(a)
    - lambda_tail * ES_alpha(-Pi_t(a))
]
```

```

- lambda_inv * I_t^2
- lambda_lat * Gamma(L_t, a)
- lambda_unc * ModelUncertainty_t(a)
]

```

Subject to:

```

product whitelist
max position
max total exposure
max daily loss
max order rate
max cancel/replace rate
margin limits
stale data halt
Rithmic session health
kill-switch state

```

This is the central production decision rule. The model maximizes risk-adjusted expected value, not raw prediction accuracy.

11. Dynamic Control Form

Value function:

$$V(t, X_t) = \sup_a E[r(X_t, a) * dt + V(t+dt, X_{t+dt}) \mid X_t]$$

Action-value form:

$$Q(X_t, a) = E[\text{risk_adjusted_future_value} \mid X_t, a]$$

$$a^*_t = \operatorname{argmax}_a Q(X_t, a)$$

The implementation may approximate Q with boosted trees, neural networks, Bayesian models, policy networks, reinforcement learning, or ensembles. The mathematical object remains a constrained stochastic-control problem.

12. Position Sizing

```

size_t(a) = min(
    product_max_size,
    strategy_max_size,
    liquidity_limit,
    margin_limit,
    daily_loss_limit,
    floor(risk_budget / ES_alpha(a)),
    floor(kappa * mu_hat_t(a) / sigma_hat_t^2(a))_+
)

```

```

mu_hat_t(a)      = expected net edge
sigma_hat_t^2(a) = variance of edge
ES_alpha(a)      = expected shortfall of action a
kappa            = fractional Kelly haircut

```

Sizing is separate from signal generation. Directional correctness cannot justify oversized exposure.

13. Training Targets

Target	Definition	Purpose
y_return	Forward mid return at 100ms, 250ms, 500ms, 1s, 5s, 15s, 60s	Direction and magnitude
y_fill	Whether candidate order fills within horizon	Queue realism
y_pnl	Fill-adjusted realized PnL after fees/slippage	Economic objective
y_AS	Adverse selection after fill	Toxic fill detection
y_stop	Probability of stop-out	Loss control

Target	Definition	Purpose
y_tail	Expected shortfall / tail loss	Risk sizing and gating
y_action	Best action under replay assumptions	Policy learning

14. Multi-Objective Training Loss

```

Loss(theta) =
    w1 * L_return
    + w2 * L_fill
    + w3 * L_pnl
    + w4 * L_adverse_selection
    + w5 * L_tail
    + w6 * L_calibration
    + Omega(theta)

```

$\Omega(\theta) = \text{regularization} / \text{complexity penalty}$

A black-box model is allowed, but the training objective must be economically grounded and validated against execution realism.

15. Required Feature Families

```

aggressor_volume_imbalance
buy_aggressor_volume / sell_aggressor_volume
cancel_to_add_ratio
near_touch_cancel_pressure
top_1 / top_3 / top_5 / top_10 depth
queue_depletion_rate
refill_speed / refill_ratio
spread_stress
book_slope / book_slope_change
absorption_score
iceberg_reload_score
liquidity_vacuum_score
lead_lag_ms
micro_vs_mini_lag
rates_equity_impulse_score
adverse_selection_after_fill
fill_probability
cancel_burst_score
stop_run_score
forced_flatten_score
cutoff_pressure_score
prop_reentry_score
news_restriction_flatten_score
realized_latency_state
volatility_state
liquidity_state
regime_state

```

16. Candidate Hypothesis Families

These are research hypotheses and feature families, not mandatory hand-coded final strategies. Each must be represented as a testable research card with features, labels, latency sensitivity, queue/fill assumptions, failure modes, and pass/fail results.

#	Hypothesis family	Core microstructure behavior
1	Second-wave continuation	Initial sweep followed by continued same-side aggressor flow
2	Stop-run exhaustion fade	Sweep through level, stop-flow spike, aggressor deceleration, opposite-side refill
3	Liquidity vacuum continuation	Top 3-10 levels vanish and fail to refill
4	Depth-refill imbalance	One side rebuilds faster and more persistently after shock
5	Spread blowout/recompression	Spread widens then compresses with improving depth

#	Hypothesis family	Core microstructure behavior
6	Aggressor deceleration fade	Market orders continue but price impact falls
7	Forced liquidation cascade	One-way flow, widening spread, repeated queue depletion, weak refill
8	False breakout trap	Price breaks level but MBO follow-through fails
9	Cancel storm before move	Large near-touch cancels before price break
10	Queue depletion trigger	Best bid/ask drains faster than refill
11	Book slope collapse	Depth disappears across top levels
12	Absorption fade	Aggressors hit level repeatedly, price fails to move
13	Iceberg/reload detection	Displayed level repeatedly trades and replenishes
14	Liquidity defense break	Large defended level stops reloading
15	One-sided add/cancel imbalance	Adds stack one side while cancels remove opposite side
16	ES -> MES lead-lag	ES pressure leads MES reaction
17	NQ -> MNQ lead-lag	NQ pressure leads MNQ reaction
18	ES/NQ divergence snapback	Short-term ES/NQ correlation gap opens and reverts
19	ZN/ZB -> ES/NQ macro impulse	Rates futures react first around macro event
20	Micro contract retail lag	Main contract leads micro contract reaction
21	Round-number stop sweep	Stop flow near obvious handles and prior highs/lows
22	Prior high/low breakout trap	Breakout lacks depth/refill/aggressor confirmation
23	Opening candle chase	09:30 ET impulse followed by late chase/reversal
24	VWAP defense/break	Repeated reload near VWAP, then hold or failure
25	DOM illusion trap	Large displayed size cancels before price reaches it
26	Late candle entry fade	Visible candle expansion with weakening aggressor flow
27	Stop-loss cascade continuation	Stops convert to market orders and book stays thin
28	Panic market-order spread tax	Forced orders cross widened spread during stress
29	End-of-day forced flatten flow	Directional exit pressure into known flat windows
30	Cutoff panic exits	Urgent flattening in final minutes before cutoff
31	No-overnight inventory squeeze	Late-day one-way reduction in exposure
32	Daily loss-limit defense	Sudden flattening after adverse move
33	Trailing drawdown pressure	Panic exit after reversal threatens account state
34	Profit-lock behavior	Trend-day winners close into cutoff window
35	Max-contract crowding in MES/MNQ	Micros show noisier forced flow than ES/NQ
36	Prop reset/reopen window	Re-entry behavior after allowed session resumes
37	Friday/weekend de-risking	Friday final-hour exposure reduction
38	Economic-event restriction flattening	Pre-news flattening and post-news re-entry
39	Quote pull before volatility	Depth vanishes before scheduled high-impact event
40	Re-quote race after shock	Depth returns unevenly after event shock
41	Thin-book continuation	Market makers remain wide after first move
42	Passive trap fill	Passive fills followed by adverse selection
43	Rebate trap avoidance	Maker fills lose more to adverse selection than benefit
44	Spread regime change	Spread remains elevated longer than baseline

17. Historical Data and Event Windows

Primary historical study: 2018 through present. Expansion: June 2017 through present where data quality is acceptable.
Live/sim shadow: latest 60 CME trading days.

Event family	Anchor	Tight	Main	Context	Symbols
CPI	08:30 ET	08:29:30-08:35 ET	08:25-08:45 ET	08:00-09:30 ET	MES, MNQ, ES, NQ, ZN, ZB
NFP	08:30 ET	08:29:30-08:35 ET	08:25-08:45 ET	08:00-09:30 ET	MES, MNQ, ES, NQ, ZN, ZB
FOMC statement	14:00 ET	13:59:30-14:05 ET	13:55-14:15 ET	13:30-15:00 ET	MES, MNQ, ES, NQ, ZN, ZB
FOMC press	14:30 ET	14:29:30-14:35 ET	14:25-14:45 ET	14:25-15:30 ET	MES, MNQ, ES, NQ, ZN, ZB
Cash open	09:30 ET	09:29:30-09:35 ET	09:25-09:45 ET	09:15-10:00 ET	MES, MNQ, ES, NQ
Topstep-style flat	15:10 CT	15:05-15:12 CT	14:45-15:20 CT	14:30-15:30 CT	MES, MNQ, ES, NQ
MyFunded-style flat	15:10 CT	15:05-15:12 CT	14:45-15:20 CT	14:30-15:30 CT	MES, MNQ, ES, NQ
Apex-style late flat	15:59 CT	15:50-16:00 CT	15:30-16:00 CT	15:00-16:00 CT	MES, MNQ, ES, NQ
Friday de-risk	Friday PM	--	14:30-16:00 CT	14:30-16:00 CT	MES, MNQ, ES, NQ

18. Databento Credit and Cost Mathematics

For request i:

```
cost_i = metadata.get_cost(  
    dataset = "GLBX.MDP3",  
    schema  = "mbo",  
    symbols = symbols_i,  
    start   = start_i,  
    end     = end_i  
)  
  
total_used      = sum(cost_i)  
credit_remaining = 125.00 - total_used  
  
duration_seconds_i = end_i - start_i  
symbol_count_i     = len(symbols_i)  
  
cost_per_symbol_second_i = cost_i / (symbol_count_i * duration_seconds_i)  
  
cost_per_symbol_minute_i = cost_i / (symbol_count_i * duration_seconds_i / 60)  
  
forecast_cost = median(cost_per_symbol_minute by event_type)  
               * planned_symbol_count  
               * planned_total_minutes
```

Manifest fields must include event_id, event_type, symbols, UTC start/end, duration, cost, cumulative cost, remaining credit, cost per symbol-minute, request hash, file path, resolved contract, and instrument ID.

19. Validation Framework

Validation layer	Requirement
Splits	Discovery 2018-2020, confirmation 2021-2022, holdout 2023-2024, recent holdout 2025-present
Walk-forward	Time-ordered only; no random split across time
Leakage audit	No feature or label can use data after decision time t
Latency	0.5ms, 1ms, 2ms, 5ms, 10ms, then empirical Rithmic distribution
Queue/fill	LogProbQueueModel2, SquareProbQueueModel, NoPartialFillExchange, empirical fill stats
Costs	Fees, spread crossing, missed fills, slippage, adverse selection

Validation layer	Requirement
Multiple tests	Deflated Sharpe or equivalent penalty; track strategy count and trials
Risk	Expected shortfall, p95/p99 drawdown, position and daily-loss gates
Production	Rithmic sim shadow before any live escalation

20. Production Failure States

Hard halts:

- stale market data halt
- Rithmic disconnect halt
- clock drift halt
- latency spike halt
- missing fill reconciliation halt
- order reject escalation
- position mismatch halt
- margin warning halt
- daily loss halt
- emergency flatten

Each halt logs:

- halt reason
- timestamp
- open orders
- current position
- attempted action
- operator alert
- recovery requirement

21. Final Production Model Interface

Input:

X_t = current market/book/queue/inventory/regime/latency/event state

Output for every valid action a :

- $EV(a)$ = expected net edge
- $P_{fill}(a)$ = fill probability
- $AS(a)$ = adverse-selection estimate
- $ES(a)$ = expected shortfall
- $Q(a)$ = risk-adjusted action value
- $size(a)$ = approved position size

Final decision:

$a^*_t = \operatorname{argmax}_a Q(X_t, a)$
 $n^*_t = size(a^*_t)$

No-trade must remain an eligible action.

22. Developer Deliverables

Package	Deliverables
1. Bare-metal build	Linux install, CPU isolation, process pinning, IRQ tuning, clock sync, security hardening, Rithmic latency report
2. Rithmic gateway	Market data listener, order entry, cancel/replace, fills/rejects, reconnect, heartbeat, position state, emergency flatten
3. Databento data system	Event calendar, cost estimator, downloader, manifest.json/parquet, UTC validation, contract roll metadata
4. HftBacktest pipeline	DBN to NPZ conversion, queue models, latency bands, empirical latency support, replay runner
5. Feature/hypothesis research	Core MBO features, 44 hypothesis families, research cards, ranking board
6. Model/decision engine	Training framework, black-box support, target construction, leakage controls, walk-forward validation, artifact registry
7. Risk readiness	Max position/loss/order-rate/cancel-rate gates, stale-data halt, disconnect halt, position mismatch handling

Package	Deliverables
8. Reporting	Latency, PnL, fill quality, adverse selection, event comparison, model ranking, deploy/sim/reject board

23. Acceptance Criteria

The project is complete only when:

1. Bare-metal server is configured and latency-tested.
2. Rithmic sim/live connectivity works from the server.
3. Databento downloader works with cost controls.
4. Event windows are reproducible from events.csv.
5. HftBacktest conversion works from raw .dbn.zst to .npz.
6. Contract roll handling is reproducible and audited.
7. MBO features are computed without leakage.
8. All 44 hypothesis families are represented as research modules.
9. Model training supports transparent and black-box models.
10. Latency bands are tested at 0.5, 1, 2, 5, and 10 ms.
11. Empirical Rithmic latency model is produced after samples exist.
12. Models are ranked by net expectancy, fill quality, latency sensitivity, and adverse selection.
13. MES/MNQ sim testing is completed before ES/NQ live consideration.
14. Risk layer can block, flatten, and halt trading.
15. Final model approval requires holdout success and Rithmic sim-shadow confirmation.

Appendix: Mathematical Foundation

The model structure is grounded in probability, stochastic processes, statistics, econometrics/time series, numerical methods, optimization/control, finance, and production computer systems. These disciplines are the required backbone for deriving models, estimating them from noisy data, computing them robustly, and implementing them in a production trading environment.